

AI-Powered Intrusion Detection System

Complete Working Mechanism & Documentation

Project: Multi-Layer IoT IDS with Real-Time Dashboard | Tech: Python (FastAPI + Scapy + XGBoost) + React.js | March 2026

1. System Overview

The AI-IDS Dashboard is a real-time network intrusion detection system that monitors all network traffic on the host machine, classifies each packet as either benign (normal) or malicious (attack), and displays the results on a live web dashboard. It combines Machine Learning models with heuristic pattern-matching rules to detect various types of cyber attacks as they happen.

The system consists of 4 main components working together:

- Network Interface - Captures all IP packets flowing through the machine
- Python Backend (FastAPI) - Analyses packets using ML + heuristics in real-time
- WebSocket - Streams classification results instantly to the browser
- React Dashboard - Displays live traffic, charts, alerts & threat levels

2. How It Works - Step by Step

Step 1: Packet Capture (Scapy)

The backend uses Scapy AsyncSniffer to capture every IP packet that flows through the machine network interface. This runs in passive (read-only) mode - it only observes traffic, never modifies or injects packets. The sniffer requires Administrator privileges to access the raw network interface.

Step 2: Rolling Buffer & Feature Extraction

Each captured packet is added to a rolling buffer of the last 30 packets. This buffer provides context - instead of looking at each packet in isolation, the system examines the recent traffic pattern. From this buffer, 6 key features are extracted:

#	Feature	What It Measures	Why It Matters
1	packet_rate	Packets per second	Attacks generate much higher rates
2	unique_dst_ips	Different destination IPs	Scans hit many IPs; normal goes to few
3	unique_src_ips	Different source IPs	Botnets spoof many fake sources
4	mean_pkt_size	Average packet size (bytes)	DoS=tiny packets; theft=large packets
5	std_pkt_size	Variation in packet sizes	Attacks have uniform sizes
6	protocol_count	Number of protocols used	Anomalies mix protocols unusually

Step 3: Classification (Heuristic Rules + ML Model)

The system uses a two-layer classification approach:

Layer 1 - Heuristic Rules (Primary): These are pattern-matching rules specifically designed to detect known attack signatures. They check combinations of the 6 features against known attack profiles. This is the primary detection

mechanism because it provides instant, reliable results.

Layer 2 - XGBoost ML Model (Secondary): An XGBoost classifier trained on the BoT-IoT dataset provides a secondary anomaly probability. If heuristic rules do not trigger but the ML model detects elevated anomaly probability combined with unusual traffic rates, it flags the traffic as suspicious.

Why Both? The ML model alone is too conservative. The heuristic rules catch specific attack patterns. Together, they provide robust detection coverage.

Step 4: Real-Time Broadcast (WebSocket)

Every classified packet is instantly sent to all connected dashboard clients via WebSocket. The system broadcasts approximately 3 packets per second (throttled to prevent browser overload). Each broadcast includes: source IP, destination IP, protocol, port, packet size, classification label, confidence score, and severity rating.

Step 5: Dashboard Visualization (React)

The React frontend receives each packet and updates all dashboard components in real-time: Stats Bar (running counts), Threat Severity Index (0-100 gauge), Network Throughput Chart (live graph), Attack Intelligence Panel (attack breakdown), and Live Packet Log (scrolling classified packets).

3. Attack Types & Detection Methods

The IDS can detect and classify the following attack types. Each has a specific traffic pattern that the system recognizes:

3.1 DoS - SYN Flood Attack

Property	Details
What It Is	Flood of TCP SYN packets without completing handsh
Attack Script	trigger_dos.py
Traffic Pattern	Very high rate (>60 pps), tiny packets (~40B), TCP
How IDS Detects	SYN count > 10 AND packet rate > 20 pps in buffer
Dashboard Label	DoS-SYN or DoS-TCP
Severity	85-90 / 100

3.2 OS / IP Sweep Scan

Property	Details
What It Is	Scans range of IPs to discover alive hosts using I
Attack Script	trigger_scan.py
Traffic Pattern	ICMP to many dest IPs (192.168.1.x), ~10 pps, ~42B
How IDS Detects	ICMP pkts > 30% of buffer AND unique dest IPs > 4
Dashboard Label	OS SCAN
Severity	60-65 / 100

3.3 DDoS - Botnet Attack

Property	Details
----------	---------

What It Is	Multiple spoofed sources attack single target simu
Attack Script	trigger_botnet.py
Traffic Pattern	Many source IPs (10.0.0.x), large pkts (~540B), 1
How IDS Detects	Unique source IPs > 8 AND mean pkt size > 300 byte
Dashboard Label	DDoS-BOTNET
Severity	92 / 100

3.4 DDoS - UDP/TCP Flood

Property	Details
What It Is	Volumetric attack flooding target with massive tra
Traffic Pattern	Extremely high rate, one protocol dominant, spoofe
How IDS Detects	UDP > 70% of buffer AND rate > 50, OR srcs > 5 AND
Dashboard Label	DDoS-UDP or DDoS-FLOOD
Severity	82-85 / 100

3.5 Service Scan (Port Scan)

Property	Details
What It Is	Probes many ports on target to find open services
Attack Script	trigger_svc_scan.py
Traffic Pattern	TCP SYN to many different ports, moderate rate, 1
How IDS Detects	High SYN count + moderate packet rate = probing pa
Dashboard Label	SVC SCAN
Severity	52-55 / 100

3.6 Data Exfiltration / Theft

Property	Details
What It Is	Stolen data sent to external server controlled by
Attack Script	trigger_theft.py
Traffic Pattern	Large pkts (~1200B), high rate, single external de
How IDS Detects	High rate + large mean pkt size + single destinati
Dashboard Label	DATA EXFIL
Severity	95 / 100

3.7 L2 Anomaly (Mixed Protocol Attack)

Property	Details
What It Is	Abnormal traffic mixing protocols at high rate

Attack Script	trigger_l2_anomaly.py
Traffic Pattern	Mix of TCP+UDP+ICMP, ~125 pps, small packets
How IDS Detects	Rate > 80 pps + mean size < 200B, or ML anomaly >
Dashboard Label	L2 ANOMALY
Severity	65-72 / 100

4. Machine Learning Models

Model	File	Purpose
XGBoost Multi-Class	bot_iot_xgb_multiclass.joblib	Classifies into DDoS/DoS/Scan/Theft/Normal
Label Encoder	label_encoder.joblib	Maps numeric predictions to labels
Layer2 Anomaly	layer2_anomaly_model.joblib	Binary anomaly detection on 6 features

Layer2 Model Feature Importance:

unique_dst_ips: 31.9% | protocol_count: 23.8% | packet_rate: 19.7% | mean_pkt_size: 15.0% | std_pkt_size: 8.5% | unique_src_ips: 1.1%

5. Technology Stack

Backend

- Python 3.11 - Core programming language
- FastAPI - Async web framework for REST API and WebSocket
- Scapy - Low-level packet capture library (raw network interface)
- XGBoost - Gradient-boosted decision tree ML model
- Pandas / NumPy - Data processing and feature extraction
- Uvicorn - ASGI server running the FastAPI application

Frontend

- React.js - Component-based UI framework
- Recharts - Charting library for live throughput graphs
- WebSocket API - Browser-native real-time communication
- Vite - Build tool for development and production
- CSS - Custom dark-themed SOC design

6. Demo Guide - Running Attack Scripts

IMPORTANT: All attack scripts must be run in a separate terminal as Administrator.

Attack	Command	Dashboard Shows
--------	---------	-----------------

DoS SYN Flood	python trigger_dos.py	Red DoS-SYN, severity 85-90
IP Sweep Scan	python trigger_scan.py	Yellow OS SCAN, many dest IPs
Botnet DDoS	python trigger_botnet.py	Red DDoS-BOTNET, many src IPs
Service Scan	python trigger_svc_scan.py	SVC SCAN / DoS-SYN entries
Data Theft	python trigger_theft.py	DATA EXFIL, large packets
L2 Anomaly	python trigger_l2_anomaly.py	L2 ANOMALY, mixed protocols

Recommended Demo Flow:

- 1. Open dashboard - show normal benign traffic flowing (green entries)
- 2. Run trigger_dos.py - dashboard immediately shows red DoS alerts
- 3. Stop attack (Ctrl+C) - traffic returns to benign within seconds
- 4. Run trigger_scan.py - show OS SCAN with multiple destination IPs
- 5. Run trigger_botnet.py - show DDoS-BOTNET with spoofed source IPs
- 6. Explain Attack Intelligence panel tracking all detected attacks

7. Key Design Decisions

- Continuous Per-Packet Analysis: Each packet processed individually with rolling context buffer. No batching delays.
- Heuristic-First Approach: Rules provide immediate detection for known patterns. ML model is backup for novel anomalies.
- Single-Port Deployment: Backend + frontend served from port 8000. Only one tunnel needed for public access.
- Auto-Detecting URLs: Frontend detects localhost vs tunnel automatically. No manual configuration needed.
- Passive Monitoring: System only reads traffic (read-only). Never sends, modifies, or blocks any packets.